Multimedia Tools and Applications (2023) 82:36379–36393
https://doi.org/10.1007/s11042-023-15042-4

TP-Detect: trigram-pixel based vulnerability detection for Ethereum smart contracts

Lohith J J[1] · Anusree Manoj K[1] · Guru Nanma P[1] · Pooja Srinivasan[1]

Received: 11 May 2022 / Revised: 2 September 2022 / Accepted: 27 February 2023 /
Published online: 30 March 2023
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2023

Abstract
Smart contracts are a set of instructions or programs that are stored on the blockchain network and run when predetermined conditions are met. Ethereum smart contracts are deployed on blockchain networks. Ethereum smart contracts are immutable and are vulnerable to simple coding errors called vulnerabilities. The aim of this paper is to classify Ethereum smart contracts vulnerabilities by feature extraction using machine learning. Pixel values collected from images and trigram feature extraction were used to construct the dataset. This dataset was trained using Multilabel k Nearest Neighbours (MLkNN), Binary Relevance kNN (BRkNN), Random Forest (RF), and Naive Bayes (NB), among other machine learning methods. The Naive Bayes Method outperforms the other models in terms of F1-score among all the algorithms tested. The Naive Bayes model achieves F1-scores of 99.38% and 99.44% using Binary Relevance and Classifier Chain respectively. In terms of F1-score, the Random Forest model attained a substantial degree of performance, with F1-scores of 96.71% and 96.61% using Binary Relevance and Classifier Chain respectively. In comparison, the lazy algorithms MLkNN and BRkNN produced lower F1-scores of 88.19% and 89.71%, respectively. This suggests that using the TriPix dataset outperforms models employed in either opcode characteristics or image-based detection used in other works.

Keywords Blockchain · Machine learning · Ethereum smart contracts · Vulnerabilities in smart contracts · Feature extraction

✉ Pooja Srinivasan
poojasrinivasan18@gmail.com

Lohith J J
lohith.cse@bmsce.ac.in

Anusree Manoj K
anusreekmanoj2000@gmail.com

Guru Nanma P
nanma.p31@gmail.com

[1] Computer Science and Engineering, B.M.S. College of Engineering, Bull Temple Road, Bangalore, 560019, Karnataka, India

1 Introduction

The rise of distributed public ledger technology has created a revolution in various sectors such as healthcare [46], finance, real estate, government, and academia. One of the most popular implementations of blockchains is smart contracts [39]. Unlike traditional contracts, smart contracts are not bound by any third parties, this agreement between two or more people on the network binds and provides equal rights in the contract and any decision cannot be made without consulting all the members related to the contract.

There are many platforms or frameworks on which blockchain operates, Bitcoin [37], Ethereum [7], Hyperledger [8] to name a few. Ethereum is an open-source blockchain framework that can be used to tap all the resources of blockchain technology and that enables developers to create decentralized networks. The cryptocurrency associated with this framework is called ethers. Smart contracts [3], which are maintained on a blockchain, are programs that run when specific requirements are met. They're frequently used to automate the execution of a contract so that all parties can be certain of the outcome immediately, without the need for any intermediaries. They can also automate a workflow by starting the next phase when specific conditions are met [25]. An Ethereum smart contract requires an account (To keep a check on the balance of ethers), a transaction (To update the ethers following any activity conducted), and an Ethereum virtual machine (It is the environment used to run the smart contracts).

Artificial intelligence [14], machine learning [44], and deep learning [27], or a machine's ability to think and act, have gathered a lot of steam and have revolutionised and improved numerous sectors, and have been used in a variety of applications like pattern recognition, image forensics [11], and digital watermarking [29] as well as offered comfort and uplifted the human race. The area of machine learning has also started creating a change for Blockchain technology. The blockchain network has a lot of advantages but like friction, has disadvantages that come in as necessary evils. Machine learning has improved the way it is perceived and, also trying to solve the drawbacks faced by blockchain technology. While smart contracts may be efficient and provide absolute transparency, one of the major drawbacks of smart contracts are the vulnerabilities that come with them. The learning potential of machine learning algorithms is enormous. Blockchain can adopt these traits to become smarter. The combination of machine learning and Blockchain has the potential to improve the security of the blockchain's distributed ledger. The golden nonce [37] can be found more quickly with the help of machine learning's computing capabilities. It is observed that most of the vulnerabilities in smart contracts follow a pattern. If a certain number of smart contracts having the same vulnerability are analyzed, it is observed that the vulnerability is caused due to a certain pattern of opcodes. Machine learning can help in pattern recognition. Pattern recognition is the process of finding patterns using machine learning techniques. Thus, machine learning can be used to identify such patterns in the smart contract opcodes that cause vulnerability and detect them prior to deployment dynamically.

A vulnerability [42] can be defined as a set of keys that are used to destroy the integrity of a given smart contract. They can be known as loopholes or weaknesses associated with a given smart contract. They are keys used to destroy the integrity of contracts which not only results in major catastrophic losses but also dissolves the whole purpose of using blockchain technology as a smart contract. It can also be considered a major loss for a smart contract. Vulnerability can be of many types, with respect to this work the vulnerabilities dealt with are integer underflow [1], integer overflow [54], call stack depth attack [30], transaction ordering dependency [40], timestamp dependency [33], and reentrancy vulnerability [43].

Integer overflow is a vulnerability where the arithmetic value exceeds the buffer size. It can also be known as an overflow of values than the prescribed limit. A suitable example to understand this class of vulnerability would be typecasting unsigned to a signed type of variable. Integer underflow occurs when a given value is less than the buffer size i.e a value that is smaller and unable to fit in. An example to describe the integer underflow would be to assume the value of an unsigned integer is set to zero but when subtracted by one it is a negative integer that cannot be handled by unsigned class.

In the call stack depth vulnerability, it exceeds the number of calls that can be made to a given method. A call stack is usually bounded by 1024 frames and if this limit is exceeded it is said that the call stack depth vulnerability has occurred. The timing of uncontrolled events in a smart contract is a major source of vulnerabilities (race conditions). Race conditions can also be known as transaction order dependency. The BANCOR protocol is a well-known example of transaction order dependency. Timestamp dependency is a vulnerability where the timestamp can be altered by a miner/hacker and can be used in direct or indirect methods. The reentrancy vulnerability occurs when a method in a contract depends on another method. When the call of a particular method depends on an external contract then changes in control flow and associated data could occur. In this attack recursively the fallback function can be called by the attacker.

Smart contracts are a set of instructions or programmes that run on the blockchain network and are executed when certain circumstances are met. Smart contracts serve as the foundation for Ethereum applications. Simple coding loopholes, make Ethereum smart contracts being exposed to vulnerabilities. This work can be summarized as follows:

- Ethereum smart contract vulnerablitiles are classified using machine learning.
- The dataset was created using pixel values extracted from images and the trigram feature extraction.
- This dataset was trained using various machine learning algorithms like MLkNN [55], BRkNN [47], Random Forest(RF) [5] and Naive Bayes(NB) [24].
- To the best of our knowledge, this combinational dataset has been used for the first time in blockchain to detect vulnerabilities in smart contracts.

Organization of the paper Section 2 gives a quick summary of relevant efforts in the field of detection of vulnerabilities in smart contracts and how machine learning could be applied to blockchain. The proposed Methodology is presented in Section 3. The methodology includes smart contract collection and preprocessing, dataset preparation, and training. The datset prepartion is done using Trigram feature extraction and Pixel value extraction and this is merged to form the dataset used for training. The results of the suggested work is discussed in Section 4, and the paper concludes with the possibility of future research opportunities in Section 5.

2 Related work

Due to the numerous advantages that comes with blockchain smart contracts, they are being widely used for different types of applications. Thus detection of vulnerabilities in smart contracts is very important. Numerous tools and frameworks have been introduced in order to detect and classify vulnerabilities in smart contracts. In this section, different tools used to detect vulnerabilities in smart contracts are discussed, followed by the role of machine learning in detecting vulnerabilities in smart contracts.

2.1 Detection of vulnerabilities in smart contracts

In recent years there are many tools used to identify vulnerabilities in smart contracts. The Oyente tool [32] is used to detect or identify a given set of vulnerabilities. It is a very useful tool to detect vulnerabilities in Ethereum smart contracts. The initial implementation of the tool showed that out of 19,366 smart contracts, the tool proved that 8,833 of them are vulnerable, including the Decentralized Autonomous Organization(DAO) bug [22] which led to a 60 million US dollar loss in June 2016.

MAIAN tool [38] used to detect bugs in Ethereum smart contracts. The tool analyzes bugs in prodigal, suicidal, and greedy Ethereum smart contracts. One million contracts were analyzed using this tool, where the tool flagged 34,200 (2,365 distinct) contracts vulnerable, in 10 seconds per contract.

Securify [50] is a security scanner used to detect vulnerabilities in Ethereum smart contracts. The tool detects some form of DAO (reentrancy), locked ether, missing input validation, transaction ordering-dependent amount, receiver and transfer, unhandled exceptions, and unrestricted ether flow.

The Vandal [6] framework decompiles the Ethereum smart contract into a control flow graph. The static operations are removed and thereby making the data dependencies visible. This with a datalog specification is fed into the Souffle analysis engine for the extraction of program properties.

The EthIR [2] framework is basically an extension of the Oyente tool. This framework generates control flow graphs (CFGs) where the bytecode is represented in a rule-based manner which helps in the better understanding of the properties of the EVM code. This framework also tries to detect different vulnerabilities for the EVM bytecode.

EtherSolve [12] is a tool used to identify vulnerabilities in smart contracts. This tool extracts a control flow graph from the available bytecode of the smart contract. This work mainly focused on the reentrancy vulnerability.

Slither [16] is a static analysis framework meant to offer rich information on Ethereum smart contracts. It operates by turning Solidity smart contracts into SlithIR, which is an intermediate form. SlithIR employs the Static Single Assignment (SSA) form and a condensed instruction set to simplify analysis implementation while keeping semantic information that would otherwise be lost when Solidity is converted to bytecode. Slither enables the usage of popular programme analysis techniques such as dataflow and taint tracking. There are four key use cases for the framework: (1) automatic vulnerability detection, (2) automated code optimization opportunities identification, (3) better user understanding of contracts, and (4) code review help.

Dynamic symbolic execution, a technique that uses constraint solving to systematically explore a program's state space, is an effective way to maximise code coverage in software tests. Manticore [35] is a dynamic symbolic execution framework for evaluating binaries and Ethereum smart contracts that is free and open-source. Manticore's versatile architecture enables it to accommodate both standard and non-traditional execution settings, and its API allows users to tailor their analysis. Manticore's architecture and capabilities were utilised to detect flaws and validate the correctness of code.

Mythril [36] is a security analysis tool for the Ethereum blockchain that analyses smart contract security concerns. It provides a unique examination of smart contract vulnerabilities based on symbolic code execution. Mythril use EVM bytecode to identify cybersecurity flaws in smart contracts created for EVM-compatible blockchains like Ethereum and Tron.

It detects various vulnerabilities, including reentrancy and unprotected functions, using taint analysis and symbolic execution.

Echidna [18] is an open-source smart contract fuzzer that makes it simple to develop automated tests for detecting assertion and custom property violations. Echidna is simple to set up and does not involve any complicated configuration or contract deployment to a local blockchain. It provides timely feedback, detects a wide range of property breaches, and its default settings have been calibrated using experimental data. Echidna has been used in over ten large paid security audits to date, and input from those audits has influenced Echidna's functionality and user experience, both in terms of practical usability (e.g., smart contract frameworks like Truffle and Embark) and test generating tactics. Echidna aspires to be excellent at detecting true defects in smart contracts with minimal user input and maximum efficiency.

In Solidity, classify and implement a full taxonomy of code concerns. SmartCheck [49] is a static analysis tool that may be extended to detect them. SmartCheck tests Solidity source code against XPath patterns after converting it to an XML-based intermediate representation. The technology was tested on a large dataset of real-world contracts, with the results compared against a manual audit on three of them. The tool represents the current state of knowledge about Solidity vulnerabilities and outperforms the competition. SmartCheck has its limitations, as some issues require more advanced approaches like taint analysis or even manual auditing to detect.

It's difficult to fix issues in a smart contract once it's been implemented, regardless of the amount of money it holds. ZEUS [23] is a framework for validating and verifying the validity and fairness of smart contracts. Correctness is defined as following safe programming principles, whereas fairness is defined as following agreed-upon higher-level business logic. To swiftly verify contracts for safety, ZEUS use both abstract interpretation and symbolic model checking, as well as the power of restricted horn clauses. With 22.4K smart contracts, the ZEUS prototype for Ethereum and Fabric blockchain systems is tested. The work shows, about 94.6 percent of contracts (worth more than $ 0.5 billion in cryptocurrency) are insecure.ZEUS is sound with zero false negatives and has a low false positive rate, with an order of magnitude improvement in analysis time as compared to prior art.

Microsoft Research's F* framework [13] is a verification approach based on the F* language. The smart contract is tested for correctness by converting the Solidity code to F* language. So the binary codes of smart contracts are readily available on the Ethereum network but the source code is difficult to obtain, the binary files on the Ethereum blockchain are decompiled to F* language to identify potential vulnerabilities such as exception disorders and reentrancy vulnerabilities.

Remix [41] is a web-based IDE for creating and debugging smart contracts written in high-level programming languages like as Solidity and Vyper. Remix eliminates code errors by identifying potentially vulnerable coding patterns. Re-entrancy, timestamp dependency, and gas-expensive patterns are among the problems it may discover.

2.2 Machine learning and blockchain

RGB channels of an image and their pixel features are being used in different applications [28]. In a recent work conducted smart contracts were analyzed for vulnerabilities using machine learning [21]. The bytecode generated from the smart contracts was used to translate to RGB color code using which images were generated. The generated images were passed through a convolutional neural network (CNN) which detected features and vulnerabilities for Ethereum smart contracts.

Zhuang, Yuan, Zhenguang Liu, Peng Qian, Qi Liu, Xiang Wang, and Qinming He proposed the idea of analyzing smart contracts for detecting vulnerabilities by using graph neural networks [57]. Eshghie, Mojtaba, Cyrille Artho, and Dilian Gurov in their work proposed a machine learning framework called Dynamit which is mainly used to detect reentrancy vulnerabilities in Ethereum smart contracts [15]. ContractWard is used to detect vulnerabilities in smart contracts using machine learning. In this work, bigram features were extracted from opcodes of smart contracts which were then passed to five machine learning algorithms and two sampling algorithms to build the models [51].

Abstract-Syntax-Tree (AST) for smart contracts was a work where a kNN model was used to detect vulnerabilities like Re-entrancy, Arithmetic, Access Control, Denial of Service, Unchecked Low-Level Calls, Bad Randomness, Front Running, and Denial of Service [53]. In the machine learning model for Smart Contracts Security Analysis work more than 1000 smart contracts were analyzed for vulnerabilities resulting in an average accuracy of 95% [34]. In the attention-based machine learning approach, a CNN model was used to detect three classes of vulnerabilities and showed that the machine learning approach was faster than the static method to detect vulnerabilities [48]. In a particular work to detect the has_short_address, has_flows, and is_greedy vulnerabilities a slicing matrix approach combined with machine learning was used. This work proved that slice matrix and neural network provide better accuracy than neural networks and opcodes combined together [52].

Eth2Vec [4] is a smart contract vulnerability detection tool that uses machine learning to detect vulnerabilities in a given contract. It takes code as input and detects vulnerabilities using neural networks and natural language processing.

3 Methodology

This work proposes a three-step approach to automatically detect vulnerabilities accurately in smart contracts. The complete process is illustrated in Fig. 1. The first step involved collecting and preprocessing smart contracts. The second step was the dataset creation, for this work the dataset was created using the following methodology: (a) The smart contracts were compiled using solc compiler to get their respective opcodes, (b) The opcodes were grouped as a sequence of three and frequency of these sequence occurrence was recorded (c) The opcodes were used as input to generate images and extraction of pixel values. Finally, different Multi-Label classification models were trained on this dataset, to classify the vulnerabilities in smart contracts. This method of using the combination of n-gram features and pixel features of the image matrix equivalent of the opcodes was adopted in Microsoft Malware Classification and feature fusion proved to give better results compared to other traditional methods of classification [56].

3.1 Collection and preprocessing of smart contracts

SmartBugs [17] is a dataset that provides a variety of smart contracts and supports different tools like Oyente, MAIAN, Securify, and many others. It is a framework for the execution of smart contracts. In this work smart contracts were collected from Smartbugs which contains 47,587 Ethereum smart contracts that were labeled according to the vulnerabilities present and were used as the source of data for both methodologies in dataset creation. Smartbugs represent 7 different types of vulnerabilities present in the Ethereum smart contracts: Integer Overflow, Integer Underflow, Parity Multisig Bug 2, Callstack Depth Attack Vulnerability, Re-entrancy Vulnerability, Transaction Ordering Dependency, and Timestamp Dependency

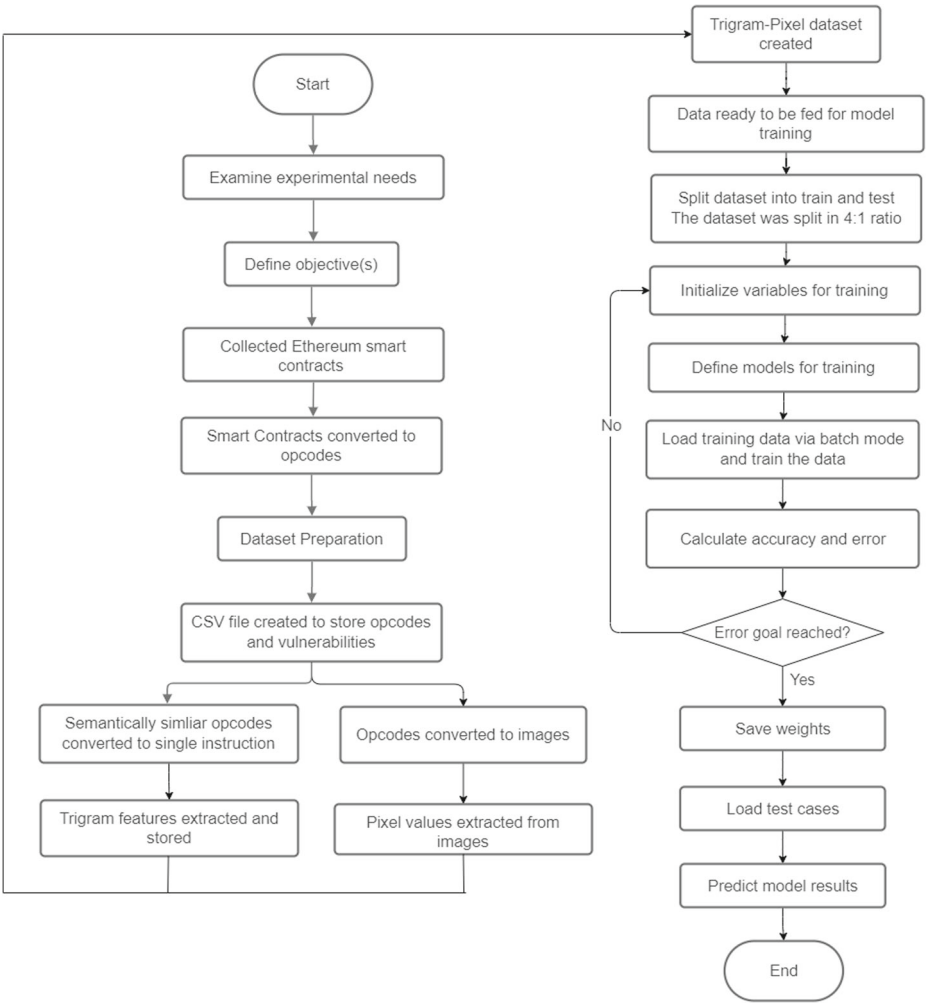


Fig. 1 Complete flow diagram of TP-Detect

Vulnerability. This dataset was preprocessed using python scripts which converted the solidity code to opcodes through Solidity Compiler [20] (solc version 4.2) and were stored along with the vulnerability labels corresponding to the smart contract in a CSV file. The opcodes were pruned to remove the unnecessary comments and extra spaces, special symbols, etc., with the help of regular expressions before storing them as shown in Fig. 2.

3.2 Dataset

In this work the TriPix dataset is created by combination of 3-gram feature extraction and pixel feature extraction to classify the vulnerabilities present in the Ethereum smart contracts. The flow diagram of the dataset creation is described in Fig. 3.

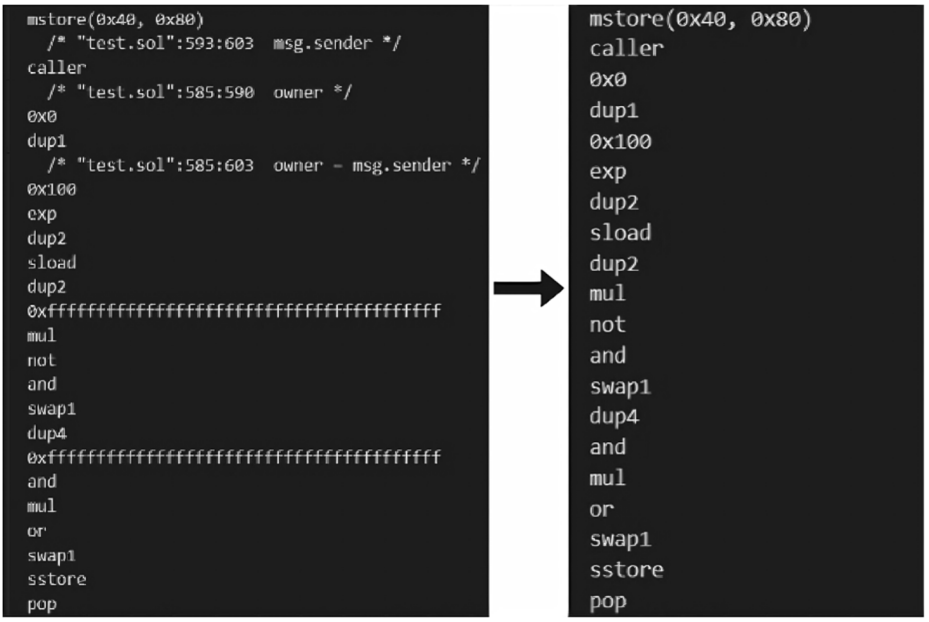


Fig. 2 Preprocessing of a compiled smart contract

3.2.1 Trigram feature extraction

When a smart contract is compiled using the solidity compiler [20] the solc compiler translates it into bytecode. A bytecode is the hexadecimal representation of a smart contract. The bytecode is converted to their respective opcodes. Opcodes are the contract's low-level, human-readable instructions. Every opcode has a hexadecimal equivalent. Among the opcodes compiled from different distinct versions of the compiler, there are several types of instructions varying semantically which were pushed onto the stack. For instance, SWAP1, SWAP2, SWAP12 [7] all have a similar method of swapping data in registers and hence were converted to a single instruction called SWAP. Similarly, the same mechanism was followed for DUPx, PUSHx, and LOGx.

N-gram [9] is a sequence of n terms from a sample of textual data or speech which are contiguous. A trigram is a special case of n-gram where the value of n is 3. In the

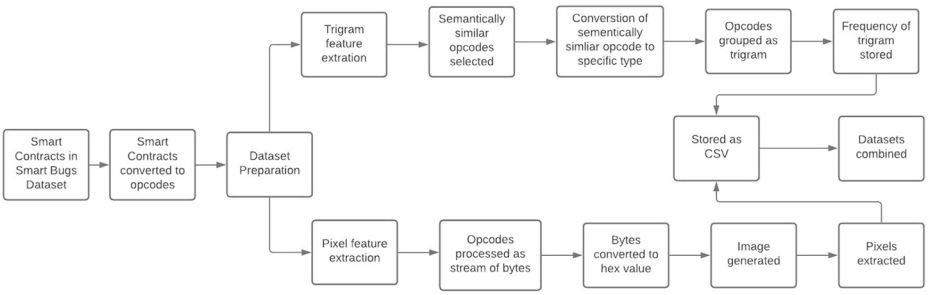


Fig. 3 Flow diagram of dataset creation

given sample a sequence of three words that are consecutive can be perceived as a trigram [26]. For the occurrence of a vulnerability in a given smart contract, the opcodes follow a certain sequence of occurrences. In this work, the frequency of all possible trigrams and their corresponding frequency of occurrence in the smart contracts are stored. This first dataset helps to classify the vulnerabilities present in the smart contracts depending on the frequency of occurrence of certain trigrams

3.2.2 Pixel value extraction

The second dataset was created by converting the opcodes generated to an image representation. This was done by reading the opcodes in a stream of bytes and converting the hexadecimal equivalent of the opcodes into a two-dimensional matrix. The generated image matrix was then represented as a grayscale image and saved in png format to be used for pixel feature extraction. Next, pixels which are digits making up images were extracted from the greyscale figures and stored in a separate dataset with their corresponding ID and vulnerabilities. The fixed size used here is 800 as it covers the entire image size.

3.2.3 Merging of datasets

The results of the trigram feature extraction and pixel value feature extraction are merged to form the intermediate dataset. The merging of the dataset is made on the basis of contract identification number resulting in the intermediate dataset containing the trigram frequencies and the pixel values. Combining the trigram frequencies and pixel-valued datasets allowed the extraction of more features than only opcodes feature extraction as given in [10, 45]. The features dataset thus generated is combined with the preprocessed, multi-labeled smart contracts dataset containing the contract identification number and the vulnerabilities labeled either as 1 or 0 based on whether the particular vulnerability is present in the smart contract or not. The merging of the datasets is done on the basis of the contract identification number. The TriPix dataset now contains the contract id, vulnerability labels, trigram frequencies, and pixel values after dropping the columns containing all 0 or null values as it is considered redundant for the final outcome.

3.3 Training

In this work, the nature of the dataset is a multi-label dataset. A given smart contract can have more than one vulnerability. Therefore while detecting vulnerabilities in the smart contract it is important to check for all the possible vulnerabilities. The smart contracts in this work are not mutually exclusive and hence our result should give us all the possible vulnerabilities present in the given contract.

Multi-label classification can naturally be handled by certain decision tree-based algorithms in Scikit-learn. The statistical non-parametric k-nearest neighbor was used. The Multilabel k-nearest neighbor MLkNN [55] was used as it outperforms kNN [19]. The random forest [5] which is a supervised algorithm of machine learning was also used to train the dataset. The Binary Relevance kNN [47] is a multi-label classification method based on kNN. In this algorithm, only half the neighbors are assigned with labels.

Random forest classifier [5], composed of multiple decision trees, is an ensemble classification model which utilizes randomness in building each decision tree promoting uncorrelated forests utilizing the forest's predictive powers to make more accurate decisions compared to an individual decision tree. This process is shown below in Fig. 4.

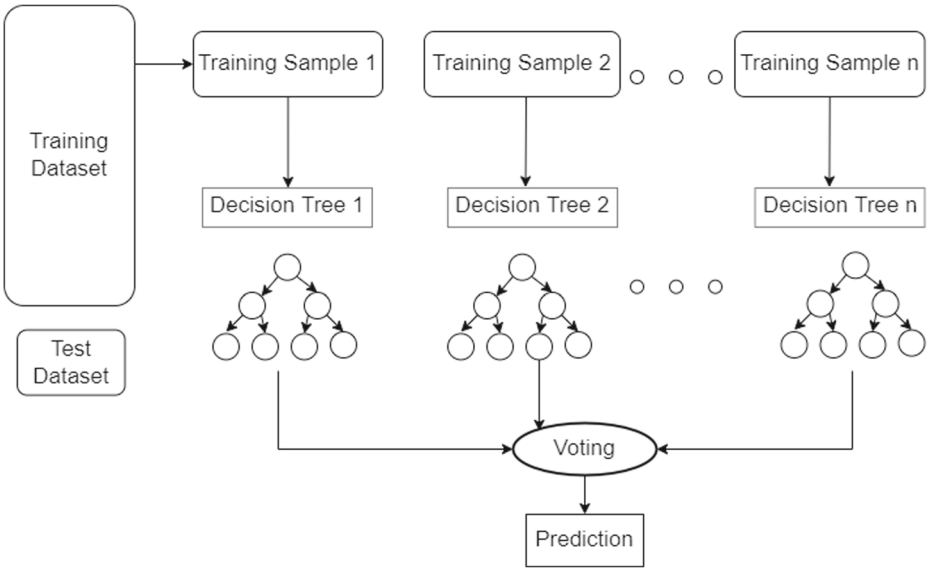


Fig. 4 Random forest classification

Naive Bayes [24] is a supervised learning algorithm based on the statistical Bayesian theorem. The conditional independence for every pair of features given the class variable. It is one of the simplest probabilistic classifiers. This algorithm assumes the existence of a feature is independent of any other feature present.

Two different training methods have been conducted with respect to multi-label training, first is the Binary Relevance method which treats each target variable independently as a distinct single class classification problem. Therefore a multi-label classification problem with M labels is converted into M different binary classification problems and finally, the results of each classifier are combined to get the output for the multi-label classification problem. The Scikit-Multilearn library automatically implements binary relevance or classifier chain as required. This ensures dividing the data separately for training and testing purposes thereby programmers are not required to divide the data into training or testing sets explicitly. The second method is the Classifier Chain method which is an optimal technique to train a multi-label classification problem. It is similar to binary relevance and makes use of its computational efficiency. It also takes into account the label dependency. The classifier chains transform the multi-label classification problem with k labels into k separate single-label classification problems. The Bayesian chain rule is followed to order the training of the k classifiers. The first classifier is trained on the input data, the second on the output of the first classifier, and so on.

4 Results

The final multi-labeled dataset containing the combination of 3-gram features and pixel features was trained using various architectures in the Scikit-Multilearn library which is compatible with multi-label classification. The dataset was divided into train and test datasets in such a way that 80% of the dataset was used for training and 20% for validation

Table 1 F1-score of the machine learing models considered

Model	Binary relevance	Classifier chain
Random forest	96.71%	96.61%
Naive bayes	99.38%	99.44%
kNN	90.80%	90.70%

by using sklearn's train-test split. F1-score was used as an evaluation metric in this work. It is the measure of this work's model accuracy on the combined final dataset based on the number of true positives and false positives predicted by the model.

$$Precision = \frac{TruePositive}{TruePositive + FalsePositive} \tag{1}$$

$$Recall = \frac{TruePositive}{TruePositive + FalseNegative} \tag{2}$$

$$F_1 = 2 * \frac{Precision * Recall}{Precision + Recall} \tag{3}$$

Table 1 shows the F1-score of the models considered in the work. Among all the algorithms tried, the Naive Bayes Model outperforms the other models in terms of F1-score. The Naive Bayes model achieves F1-scores of 99.38% and 99.44% using Binary Relevance and Classifier Chain respectively. The Random Forest model also achieved a significant level of performance in terms of F1-scores which are 96.71% and 96.61% using Binary Relevance and Classifier Chain respectively. Comparatively the experimental results demonstrated that the lazy algorithm - MLkNN and BRkNN gave lower F1-scores of 88.19% and 89.71% respectively. Figure 5 quantifies through the graph the comparative analysis of each model's F1-Score.

Among the different algorithms used in this work, Random Forest and Naive Bayes gave better F1 scores. Table 2 shows the performance comparison of existing machine learning

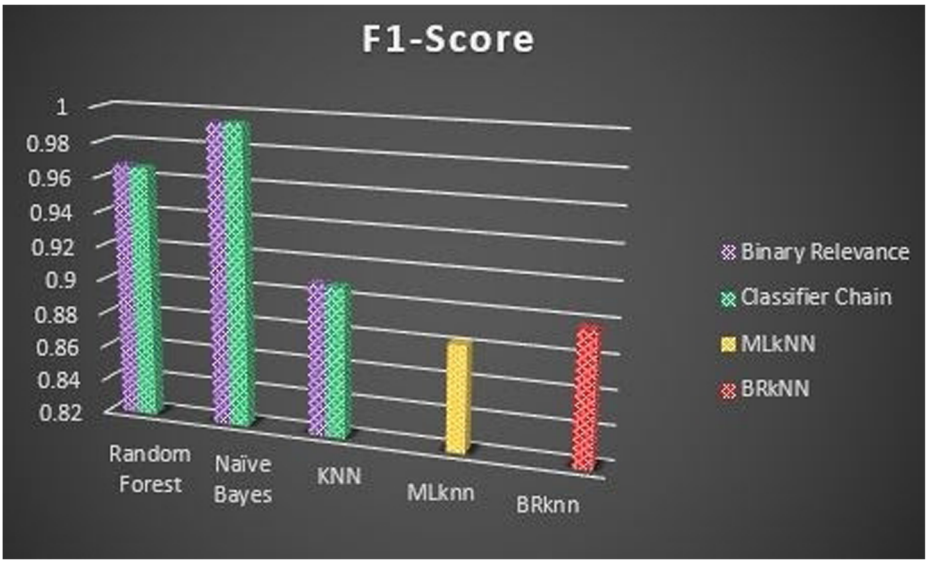


Fig. 5 The F1-Scores of each multi-label classification algorithm

Table 2 Performance comparsion of existing machine learning techniques with TP-Detect

Tool	Methodology	Evaluation metrics
Dynamit [15]	Feature extraction from transaction data using machine learning.	RF = 89% NB = 83% kNN = 75%
A novel machine learning-based analysis model for smart contract vulnerability [53]	Abstract-Syntax-Tree (AST)	kNN = 90%
Attention-based machine learning model for smart contract vulnerability detection [48]	Attention-based convolutional neural networks (ABCNN).	Average F1 Score = 87.66%
Hunting the Ethereum smart contract: color-inspired inspection of potential Attacks [21]	Transfer learning	Average F1 score=97%
ContractWard [51]	Machine learning and sampling algorithms applied on Bigram features extracted from opcodes.	Average F1 score=96%
ESCORT [31]	Deep neural networks that leverages multi output NN architecture.	Average F1 score=95%
Eth2Vec [4]	Neural networks and natural language processing.	Average F1 score = 57.50%
TP-Detect [our work]	**Machine learning architectures that train on the TriPix dataset which is a combination of trigram features extracted from opcodes and pixel values extracted from the image representation of opcodes.**	**Average F1 scores RF = 96.66% NB = 99.41% kNN = 90.75%**

techniques for detecting vulnerabilities in smart contracts with TP-Detect. This comparative analysis indicates that using TriPix dataset shows improved performance compared to the models which used either the opcode features or the image based detection.

5 Conclusion

The proposed work uses machine learning to detect Ethereum smart contract vulnerabilities. Our idea of employing machine learning provides a lot of benefits such as the entire procedure is automated, new vulnerabilities can be easily added to our model architecture as they emerge and each cycle of training improves the model. To the best of our knowledge this is one of the first approaches to such a dataset has been created by merging the Trigram feature extraction and Pixel Value Extraction. This dataset allows in-depth analysis of the vulnerable smart contracts. This has proven to be effective compared to the models based on opcode features or image-based detection used in prior works.

The proposed TP-Detect approach identifies vulnerablities in Ethereum smart contracts using multi-label classification algorithms. In the future, different deep learning models and tranfer learning methods can be applied on TriPix dataset for detecting vulnerabilities in Ethereum smart contracts. In addition, the proposed model can be used to detect impending vulnerablities identified in the future.

Data Availability The datasets generated during and analysed during the current study are available from the corresponding author on reasonable request.

Declarations

Competing interests The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

1. Ahmad D (2003) The rising threat of vulnerabilities due to integer errors. IEEE Sec Priv 1(4):77–82
2. Albert E, Gordillo P, Livshits B, Rubio A, Sergey I (2018) Ethir: a framework for high-level analysis of ethereum bytecode. In: International symposium on automated technology for verification and analysis. Springer, pp 513–520
3. Alharby M, Van Moorsel A (2017) Blockchain-based smart contracts: a systematic mapping study. arXiv: 1710.06372
4. Ashizawa N, Yanai N, Cruz JP, Okamura S (2021) Eth2vec: learning contract-wide code representations for vulnerability detection on ethereum smart contracts. In: Proceedings of the 3rd ACM international symposium on blockchain and secure critical infrastructure, pp 47–59
5. Breiman L (2001) Random forests. Mach Learn 45(1):5–32
6. Brent L, Jurisevic A, Kong M, Liu E, Gauthier F, Gramoli V, Holz R, Scholz B (2018) Vandal: a scalable security analysis framework for smart contracts. arXiv:1809.03981
7. Buterin V et al (2013) Ethereum white paper. GitHub Repository 1:22–23
8. Cachin C et al (2016) Architecture of the hyperledger blockchain fabric. In: Workshop on distributed cryptocurrencies and consensus ledgers, vol 310. IL, Chicago, pp 1–4
9. Cavnar WB, Trenkle JM et al (1994) N-gram-based text categorization. In: Proceedings of SDAIR-94, 3rd annual symposium on document analysis and information retrieval, vol 161175. Citeseer
10. Chen W, Zheng Z, Ngai EC-H, Zheng P, Zhou Y (2019) Exploiting blockchain data to detect smart ponzi schemes on ethereum. IEEE Access 7:37575–37586
11. Chen J, Liao X, Qin Z (2021) Identifying tampering operations in image operator chains based on decision fusion. Signal Process Image Commun 95:116287

12. Contro F, Crosara M, Ceccato M, Preda MD (2021) Ethersolve: computing an accurate control-flow graph from ethereum bytecode. arXiv:2103.09113
13. Dharanikota S, Mukherjee S, Bhardwaj C, Rastogi A, Lal A (2021) Celestial: a smart contracts verification framework. In: 2021 Formal methods in computer aided design (FMCAD). IEEE, pp 133–142
14. Dick S (2019) Artificial intelligence
15. Eshghie M, Artho C, Gurov D (2021) Dynamic vulnerability detection on smart contracts using machine learning. In: Evaluation and assessment in software engineering, pp 305–312
16. Feist J, Grieco G, Groce A (2019) Slither: a static analysis framework for smart contracts. In: 2019 IEEE/ACM 2nd international workshop on emerging trends in software engineering for blockchain (WETSEB). IEEE, pp 8–15
17. Ferreira JF, Cruz P, Durieux T, Abreu R (2020) Smartbugs: a framework to analyze solidity smart contracts. In: Proceedings of the 35th IEEE/ACM international conference on automated software engineering, pp 1349–1352
18. Grieco G, Song W, Cygan A, Feist J, Groce A (2020) Echidna: effective, usable, and fast fuzzing for smart contracts. In: Proceedings of the 29th ACM SIGSOFT international symposium on software testing and analysis, pp 557–560
19. Guo G, Wang H, Bell D, Bi Y, Greer K (2003) Knn model-based approach in classification. In: OTM Confederated international conferences on the move to meaningful internet systems. Springer, pp 986–996
20. Hegedűs P (2019) Towards analyzing the complexity landscape of solidity based ethereum smart contracts. Technologies 7(1):6
21. Huang TH-D (2018) Hunting the ethereum smart contract: color-inspired inspection of potential attacks. arXiv:1807.01868. Accessed 12 Oct 2021
22. Jentzsch C (2016) Decentralized autonomous organization to automate governance. White paper, November
23. Kalra S, Goel S, Dhawan M, Sharma S (2018) Zeus: analyzing safety of smart contracts. In: Ndss, pp 1–12
24. Kaviani P, Dhotre S (2017) Short survey on naive bayes algorithm. Int J Adv Res Comput Sci Manag 04
25. Khan SN, Loukil F, Ghedira-Guegan C, Benkhelifa E, Bani-Hani A (2021) Blockchain smart contracts: applications, challenges, and future trends. Peer-to-peer Netw Appl:1–25
26. Kumar R, Tripathi RC (2014) A trigram word selection methodology to detect textual similarity with comparative analysis of similar techniques. In: 2014 Fourth international conference on communication systems and network technologies, pp 383–387. https://doi.org/10.1109/CSNT.2014.82
27. LeCun Y, Bengio Y, Hinton G (2015) Deep learning. Nature 521(7553):436–444
28. Liao X, Yu Y, Li B, Li Z, Qin Z (2019) A new payload partition strategy in color image steganography. IEEE Trans Circ Syst Video Technol 30(3):685–696
29. Liao X, Peng J, Cao Y (2021) Gifmarking: the robust watermarking for animated gif based deep learning. J Vis Commun Image Represent 79:103244
30. Liljestrand H, Nyman T, Gunn LJ, Ekberg J-E, Asokan N (2021) Pacstack: an authenticated call stack. In: 30th {USENIX} security symposium ({USENIX}, security 21)
31. Lutz O, Chen H, Fereidooni H, Sendner C, Dmitrienko A, Sadeghi AR, Koushanfar F (2021) Escort: ethereum smart contracts vulnerability detection using deep neural network and transfer learning. arXiv:2103.12607
32. Luu L, Chu D-H, Olickel H, Saxena P, Hobor A (2016) Making smart contracts smarter. In: Proceedings of the 2016 ACM SIGSAC conference on computer and communications security, pp 254–269
33. Min SL, Baer J-L, Mn M (1989) A timestamp-based cache coherence scheme
34. Momeni P, Wang Y, Samavi R (2019) Machine learning model for smart contracts security analysis. In: 2019 17th International conference on privacy, security and trust (PST). IEEE, pp 1–6
35. Mossberg M, Manzano F, Hennenfent E, Groce A, Grieco G, Feist J, Brunson T, Dinaburg A (2019) Manticore: a user-friendly symbolic execution framework for binaries and smart contracts. In: 2019 34th IEEE/ACM international conference on automated software engineering (ASE). IEEE, pp 1186–1189
36. Mueller B, Honig J, Parasaram N et al (2023) ConsenSys/Mythril
37. Nakamoto S (2008) Bitcoin: a peer-to-peer electronic cash system. Decentralized Bus Rev:21260
38. Nikolić I, Kolluri A, Sergey I, Saxena P, Hobor A (2018) Finding the greedy, prodigal, and suicidal contracts at scale. In: Proceedings of the 34th annual computer security applications conference, pp 653–663
39. Parthasarathy S, Harikrishnan A, Narayanan G, J LJ, Singh K (2021) Secure distributed medical record storage using blockchain and emergency sharing using multi-party computation. In: 2021 11th IFIP International conference on new technologies, mobility and security (NTMS), pp 1–5. https://doi.org/10.1109/NTMS49979.2021.9432643

40. Ray I, Xin T (2006) Analysis of dependencies in advanced transaction models. Distrib Parallel Databases 20(1):5–27
41. Remix- Ethereum (2023) https://remix.ethereum.org/#optimize=false&runs=200&evmVersion=null&version=soljson-v0.8.7+commit.e28d00a7.js
42. Sambana B, Ramesh Y, Rao MS (2020) Blockchain approach to cyber security vulnerabilities attacks and potential countermeasures. Int J Sec Appl 14(1):1–14
43. Samreen NF, Alalfi MH (2020) Reentrancy vulnerability identification in ethereum smart contracts. In: 2020 IEEE international workshop on blockchain oriented software engineering (IWBOSE). IEEE, pp 22–29
44. Sarker IH (2021) Machine learning: algorithms, real-world applications and research directions. SN Comput Sci 2(3):1–21
45. Sezer S, Eyhoff C, Prinz W, Rose T (2020) Exploiting smart contract bytecode for classification on ethereum. In: PoEM Workshops, pp 11–22
46. Singh R, Mir BA, Chakravarthi DS, Alharbi AR, Kumar H, Hingaa SK et al (2022) Smart healthcare system with light-weighted blockchain system and deep learning techniques. Comput Intell Neurosci 2022
47. Spyromitros E, Tsoumakas G, Vlahavas I (2008) An empirical study of lazy multilabel classification algorithms. In: Hellenic conference on artificial intelligence. Springer, pp 401–406
48. Sun Y, Gu L (2021) Attention-based machine learning model for smart contract vulnerability detection. In: Journal of physics: conference series, vol 1820. IOP Publishing, p 012004
49. Tikhomirov S, Voskresenskaya E, Ivanitskiy I, Takhaviev R, Marchenko E, Alexandrov Y (2018) Smartcheck: static analysis of ethereum smart contracts. In: Proceedings of the 1st international workshop on emerging trends in software engineering for blockchain, pp 9–16
50. Tsankov P, Dan A, Drachsler-Cohen D, Gervais A, Buenzli F, Vechev M (2018) Securify: practical security analysis of smart contracts. In: Proceedings of the 2018 ACM SIGSAC conference on computer and communications security, pp 67–82
51. Wang W, Song J, Xu G, Li Y, Wang H, Su C (2020) Contractward: automated vulnerability detection models for ethereum smart contracts. IEEE Trans Netw Sci Eng
52. Xing C, Chen Z, Chen L, Guo X, Zheng Z, Li J (2020) A new scheme of vulnerability analysis in smart contract with machine learning. Wirel Netw:1–10
53. Xu Y, Hu G, You L, Cao C (2021) A novel machine learning-based analysis model for smart contract vulnerability. Sec Commun Netw 2021
54. Zeng F, Mao L, Chen Z, Cao Q (2009) Mutation-based testing of integer overflow vulnerabilities. In: 2009 5th International conference on wireless communications, networking and mobile computing. IEEE, pp 1–4
55. Zhang M-L, Zhou Z-H (2007) Ml-knn: a lazy learning approach to multi-label learning. Pattern Recognit 40(7):2038–2048
56. Zhu X, Huang J, Wang B, Qi C (2021) Malware homology determination using visualized images and feature fusion. PeerJ Comput Sci 7:494
57. Zhuang Y, Liu Z, Qian P, Liu Q, Wang X, He Q (2020) Smart contract vulnerability detection using graph neural network. In: IJCAI, pp 3283–3290

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.